

On the Optimal “Minimum Point Value”

Anderson Lai

June 15, 2026

Abstract

We aim to find the optimal “minimum point value” used in `texnique.js` to maximize the amount of points earned in light of constraining factors.

1 Introduction

Initially, this problem may be trivial. Simply, set the “minimum point value” (MPV) to the highest available point so that only the highest point problem is solved. Whilst this appears to be a valid solution, it encounters some problems:

1. Scores exceeding 2000 points are automatically rejected by the leaderboard; they can not be inputted into the leaderboard.
2. Pruning for only these high value problems results in having to skip many problems, which incurs a time penalty. Hence, fewer problems can be solved.
3. The end screen renders all the skipped problems. Since so many problems are skipped, the end screen must render an absurd amount of problems, leading to the browser tab stalling and crashing.

The main issue addressed here is the first issue. Whilst the second issue is somewhat relevant, the time penalty is minimal, and so it may be ignored. Moreover, as we will see later (or is immediately obvious), the third problem is naturally mitigated as we lower the MPV.

2 Calculations

It may be useful to have an estimate of the number of problems that the boat can solve in the timed mode (3 minutes). When setting the MPV to 13 points, an average of 96.33 problems can be solved. With the MPV at 17 points, around 93 problems can be solved (this reinforces that issue 2 can be ignored). It is also useful to have the number of questions for each point value available.¹

1. Akshay Ravikumar and Rahul, “TeXnique,” April 29, 2026, accessed June 15, 2026, <https://github.com/akshayravikumar/TeXnique.git>.

Point Value	Question Count
1	2
2	6
3	9
4	15
5	22
6	19
7	16
8	24
9	14
10	8
11	8
12	9
13	11
14	8
15	5
16	5
17	1
21	1
22	1
33	1

Table 1: Question Count by Point Value

We can now calculate the *expected value* of a problem's point value for a problem that has at least a minimum number of points. This is done by calculating the total number of questions available that meet this point threshold. Next, the relative abundance of each point value is calculated. Then, the standard expected value formula ($\mathbb{E}[X] = \sum_{i=1}^N p_i x_i$) is applied.

Minimum Point Value	Expected Value
1	8.29
2	8.37
3	8.59
4	8.89
5	9.37
6	10.10
7	10.79
8	11.43
9	12.57
10	13.43
11	13.98
12	14.55
13	15.24
14	16.36
15	17.71
16	19.22
17	23.25
21	25.33
22	27.50
33	33.00

Table 2: Expected Value for at least a Minimum Point Value

As expected, the expected value increases as the MPV increases. However, this is the expected value

per point. We are instead interested in the expected value at the end of the 3 minutes for the timed mode. Hence, we must multiply each value by 95 (an average number of points that can be solved in 3 minutes) to obtain our final expected value.

Minimum Point Value	Expected Value
1	787.7
2	795.3
3	815.8
4	844.3
5	889.8
6	959.4
7	1025.5
8	1085.6
9	1194.1
10	1275.9
11	1328.1
12	1382.0
13	1448.0
14	1554.5
15	1682.9
16	1826.1
17	2208.8
21	2406.7
22	2612.5
33	3135.0

Table 3: Expected Value After 3 Minutes

Our desired MPV immediately becomes obvious: 17. It meets the leaderboard cap of 2000 points by a margin of 200 points (10%), whilst not exceeding the cap by too much, avoiding having to skip too many problems (addressing issues 2 and 3). This margin also provides a buffer to account for the ignoring of the time-penalty issue (issue 2).

Yet, there is one small issue. Currently, the bot does not know how to stop itself when it reaches 2000 points. Furthermore, it does not know how to prune a problem with the exact number of points to reach 2000 points as the accumulated score approaches the leaderboard cap. As such, the bot will, without a second thought, readily burst past the leaderboard cap of 2000 points. Moreover, with the MPV set to 17 points, the end screen oftentimes begins to slow down and stall and is on the verge of crashing.

3 Conclusion

Therefore, with the current implementation of the bot, it is actually optimal to set the MPV at **16 points**. This ensures that the expected score is below 2000 points, so that the score can always be submitted to the leaderboard. Referring to Table 1, there are only 4 problems that are at least 17 points. By decreasing the MPV to 16 points, the number of available problems increases from 4 to 9 (a 125% increase). Thus, the quantity of problems that must be skipped is drastically reduced, addressing issues 2 and 3.

In the future, when specific pruning for the final few problems nearing 2000 total points is implemented, the MPV may be raised to 17 points. This will allow for the score to always be submitted to the leaderboard and also limit the possibility of crashing (addressing issues 1 and 3). This is because lower value problems can be solved and fewer problems must be skipped as the final part of the game may be completely skipped due to already having reached 2000 points. Hence, the loading demands of the end screen will be reduced.

References

Ravikumar, Akshay, and Rahul. “TeXnique,” April 29, 2026. Accessed June 15, 2026. <https://github.com/akshayravikumar/TeXnique.git>.